# Privacy-Preserving Dynamic Re-computation for Multi-Domain Segment Routing: An Abstract Event Notification Framework


1st Devesh Saini
*Information Technology Department*
*Netaji Subhas University of Technology*
New Delhi, India
devesh.saini.ug23@nsut.ac.in

2nd Dev Aggarwal
*Information Technology Department*
*Netaji Subhas University of Technology*
New Delhi, India
dev.aggarwal.ug23@nsut.ac.in

3rd Rohan Singh
*Information Technology Department*
*Netaji Subhas University of Technology*
New Delhi, India
rohan.singh.ug23@nsut.ac.in



***Abstract*—Inter-domain Segment Routing (SR) networks present a fundamental tension between operational privacy and path quality: Child Path Computation Elements (PCEs) must not disclose internal domain topology to a coordinating Parent PCE, yet domain-internal degradation events must trigger rapid global path re-computation. Without a real-time notification mechanism, the Parent PCE operates on a stale domain abstraction, and traffic continues to traverse degraded paths for 300–500 seconds during BGP re-convergence [10], [11]—causing systematic Service Level Agreement (SLA) violations. This paper proposes the Abstract Domain State Object (ADSO) protocol, a lightweight extension to the Path Computation Element Protocol (PCEP) that enables Child PCEs to notify a Parent PCE of domain degradation using exactly five topology-concealing abstract metrics. The proposed scheme is validated on a three-domain, 17-node emulated testbed built with Mininet and FRRouting, with Python-based Child and Parent PCE implementations communicating over a real TCP pipeline. Experimental results across four failure scenarios—link failure, ASBR degradation, complete ASBR down, and recovery—demonstrate post-notification re-convergence (from ADSO receipt to SR policy push) of 12–16 ms, approximately 25,000× faster than the BGP re-convergence baseline, while incurring approximately 2× overhead relative to a full-topology oracle. The end-to-end latency from failure occurrence is dominated by the configurable Child PCE polling interval (default ∼4 s in the prototype; reducible to <100 ms in production). The results confirm that near-oracle post-notification performance is achievable under complete topological privacy.**



***Index Terms*—Segment Routing, Multi-Domain Networks, PCE, PCEP, Privacy-Preserving Networking, SR-MPLS, SRv6, Network Resilience, Path Computation, Hierarchical PCE, Traffic Engineering**


## I. Introduction

The modern Internet is structured as a federation of independently administered networks, each referred to as an Autonomous System (AS) or routing domain. These domains treat their internal topology, link capacity, and routing policies as strictly confidential information. While intra-domain routing protocols such as IS-IS [14] and OSPF [15] provide sub-second topology visibility within a single domain, the principal mechanism for exchanging reachability information between domains remains the Border Gateway Protocol (BGP) [1], whose convergence timescale is measured in minutes [10].

Segment Routing (SR) [2] has emerged as a source-based path encoding paradigm for traffic engineering. The ingress router encodes the complete forwarding path as an ordered list of Segment Identifiers (SIDs), eliminating per-flow state at transit nodes. SR may be instantiated over an MPLS data plane (SR-MPLS [3]) or over IPv6 (SRv6 [4]). For multi-domain SR deployments, the Hierarchical PCE (H-PCE) architecture [6] assigns each domain a Child PCE that maintains complete internal topology visibility, while a Parent PCE coordinates cross-domain SR policies using a coarse, abstract view of each domain.

### *The Stale Abstraction Problem*

A fundamental and previously unsolved challenge in the H-PCE model is the *stale abstraction problem*. When a failure or degradation event occurs inside a domain—a core link fails, an Autonomous System Border Router (ASBR) becomes congested, or a border router goes offline entirely—the Parent PCE's abstract domain model becomes instantly incorrect. Because no standard mechanism exists for the Child PCE to communicate this state change to the Parent PCE in a topology-concealing manner, the Parent PCE continues to steer traffic through the degraded domain, unaware that its path selection is now sub-optimal or completely broken. Traffic remains on the degraded path until BGP eventually reconverges—a process that, depending on BGP timer configurations and the number of affected prefixes, can require between five minutes and over eight minutes [10], [11]. For latency-sensitive workloads such as real-time video conferencing, financial transaction processing, or industrial control traffic with sub-100 ms SLA requirements, this delay is operationally unacceptable.

### *Limitations of Existing Solutions*

Prior work has addressed adjacent aspects of the problem but no existing protocol satisfies all requirements simultaneously. RFC 5520 [7] introduced Path-Key Confidentiality but provides no mechanism for post-computation state notifications. RFC 8231 [8] extended PCEP for stateful path computation but requires full topological knowledge, violating

H-PCE confidentiality. BGP-LS [9] enables topology export to SDN controllers but inherits BGP's slow convergence and exposes internal topology. Static abstract link metrics [13] go stale immediately upon any network event. TI-LFA [16] provides local fast-reroute but operates only within a single domain with no cross-domain signalling.

*Proposed Solution and Contributions*

This paper proposes the **Abstract Domain State Object (ADSO)** protocol—a new PCEP TLV that enables Child PCEs to carry five carefully chosen aggregate metrics in a standard PCEP PCNtf (Notification) message, triggering the Parent PCE to recompute and push a new SR policy within milliseconds of a domain event, while preserving complete topology confidentiality. The key contributions are:

1) A specification of the ADSO TLV encoding with five topology-concealing abstract metrics and their privacy properties.
2) A threshold triggering mechanism—absolute delay, relative bandwidth drop, binary ASBR reachability, and absolute packet loss—with symmetric recovery detection and a per-domain 500 ms rate limiter.
3) A weighted cost-function re-computation engine that assembles cross-domain SR segment lists from per-domain SRGBs.
4) Experimental validation on a three-domain, 17-node emulated testbed with a real TCP-based ADSO pipeline between Python Child and Parent PCE implementations, demonstrating 12–16 ms re-convergence.
5) Comparative analysis of SR-MPLS and SRv6 recomputation latency scaling with domain count.

The remainder of this paper is organised as follows. Section II surveys related work. Section III describes the proposed protocol design, system model, topology, and testbed implementation. Section IV presents and analyses the experimental results. Section V concludes and outlines future work.

## II. Literature Survey

### A. Segment Routing and Traffic Engineering

Filsfils et al. [2] introduced Segment Routing as a source-based, stateless forwarding paradigm that unifies traffic engineering and fast reroute under a single data-plane mechanism, eliminating the signalling complexity and per-flow state maintenance of RSVP-TE. The SR-MPLS variant [3] encodes SIDs as 20-bit MPLS labels, retaining hardware compatibility with all existing MPLS-capable platforms. SRv6 [4] encodes SIDs as 128-bit IPv6 addresses in a Segment Routing Header extension, enabling richer network programming at the cost of header overhead that scales with path length (typically 40–80 bytes per additional SID). Lebrun et al. [17] demonstrated a full SRv6 implementation in the Linux kernel, validating sub-microsecond per-hop processing overhead on commodity hardware. Ventre et al. [18] provided a comprehensive survey of SR-based traffic engineering deployments, noting the lack of a standardised cross-domain dynamic re-optimisation mechanism as an open research challenge.

### B. Hierarchical PCE Architecture

The Path Computation Element Protocol (PCEP) [5] defines a client-server request-response model enabling a router acting as a Path Computation Client (PCC) to request optimal paths from a PCE server. The foundational H-PCE architecture [13] extended this model to multiple administrative domains, assigning domain-level Child PCEs that report to a coordinating Parent PCE. RFC 8685 [6] formalised the hierarchical architecture in an operational context but explicitly deferred the question of how Child PCEs should communicate dynamic domain state changes to the Parent PCE to future work. Farrel and King [12] examined the applicability of PCEP to SR networks, noting both the promise of the H-PCE model for multi-domain SR and the absence of a real-time, privacy-preserving state notification mechanism. Zhang et al. [20] surveyed PCE-based architectures in SDN environments, identifying the stale abstraction problem as one of the most significant open challenges for production multi-domain deployments.

### C. Path Confidentiality Mechanisms

Bradford et al. [7] introduced Path-Key Confidentiality for the H-PCE architecture, enabling a Child PCE to return an opaque key to the Parent PCE in place of explicit internal path segments. The path-key is later expanded to a concrete path only when the packet actually traverses that domain. While this mechanism effectively protects the internal path structure during computation, it addresses only the initial path computation and provides no mechanism for notifying the Parent PCE that the path associated with a given path-key has been subsequently degraded, invalidated, or rerouted. This is precisely the operational gap that ADSO is designed to fill.

### D. Stateful PCE and PCEP Extensions

Crabbe et al. [8] defined extensions to PCEP for stateful path computation, enabling PCEs to maintain a live database of label-switched path (LSP) state and to initiate or update paths proactively through PCInitiate and PCUpdate messages. Subsequent work [12] extended stateful PCEP to SR networks, enabling segment list computation and push for both SR-MPLS and SRv6 paths. However, stateful PCEP was designed for intra-domain use: a stateful Parent PCE in the H-PCE model would require full internal topology from each domain to maintain accurate LSP state—a direct violation of the confidentiality model. ADSO complements stateful PCEP by providing the notification substrate that triggers the Parent PCE to issue PCUpdate messages while remaining topology-agnostic.

### E. Fast Reroute and BGP Convergence

Topology-Independent Loop-Free Alternates (TI-LFA) [16] provides a local fast-reroute mechanism for SR networks that activates within 15–50 ms of a failure, using pre-computed backup SID stacks to reroute traffic around the failed element. TI-LFA operates entirely within a single domain and at the data-plane level, independent of the PCE control plane. In the ADSO architecture, TI-LFA serves as the first-line defence:

it provides immediate local repair while the ADSO notification propagates to the Parent PCE and a globally optimised replacement policy is computed and pushed.

Labovitz et al. [10] conducted a comprehensive measurement study of BGP convergence behaviour, demonstrating that inter-domain routing convergence after a topology change typically requires 3–8 minutes due to BGP's MRAI (Minimum Route Advertisement Interval) timer, path exploration, and route dampening. These convergence delays persist in modern deployments despite protocol improvements, making BGP unsuitable as a real-time failure notification mechanism for SR path re-computation.

### F. Summary of the Research Gap

Table I positions ADSO against existing approaches. No prior standard satisfies all four requirements—topology privacy, sub-second notification, rich metrics, and rate control—simultaneously.

TABLE I
COMPARISON OF EXISTING APPROACHES AGAINST ADSO REQUIREMENTS

| Approach | Topology Privacy | Sub-50ms Notif. | Rich Metrics | Rate Control |
|---|---|---|---|---|
| RFC 5520 [7] | ✓ | × | × | × |
| RFC 8685 [6] | ✓ | × | × | × |
| BGP-LS [9] | × | × | ✓ | × |
| Stateful PCE [8] | × | ✓ | ✓ | × |
| TI-LFA [16] | ✓ | ✓ | × | × |
| **ADSO (proposed)** | ✓ | ✓ | ✓ | ✓ |

## III. PROPOSED WORK

### A. System Model and Assumptions

The proposed system considers a set of $N$ autonomous domains $\mathcal{D} = \{D_1, D_2, \ldots, D_N\}$, each administered independently. Each domain $D_i$ is managed by a Child PCE $C_i$ that maintains internal topology visibility through `vtysh` queries to domain routers running FRRouting [19]. A Parent PCE $P$ coordinates global cross-domain SR policies and maintains an abstract domain model $\mathcal{M}_i$ for each domain, consisting exclusively of the five ADSO metrics.

The privacy constraint requires that ADSO notifications must not reveal: (i) the count or identity of internal links; (ii) internal IP addressing; (iii) per-link utilisation; (iv) internal node identity; or (v) IGP metric values. The system assumes each domain operates TI-LFA [16] for local repair within $\sim$15 ms.

Fig. **??** illustrates the H-PCE architecture with ADSO flow.

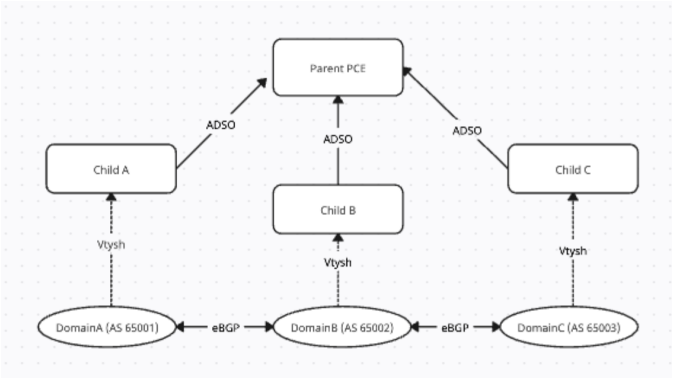


Fig. 1. Hierarchical PCE architecture with ADSO notifications. Child PCEs poll FRR routers via `vtysh` and send five-metric ADSO messages to the Parent PCE over TCP. Internal topology is never disclosed.

### B. The ADSO TLV: Five Abstract Metrics

The ADSO TLV is encoded as a new Type value inside the existing PCEP PCNtf message [5]. The TLV carries exactly five fields, defined in Table II.

TABLE II
ADSO ABSTRACT METRICS: ENCODING AND PRIVACY GUARANTEE

| # | Metric | Size | Privacy Guarantee |
|---|---|---|---|
| 1 | Min. E2E delay (ms) | 4 B | Domain aggregate; no link identity |
| 2 | Max. bandwidth (Mbps) | 4 B | Supremum over paths; no path info |
| 3 | Max. SR SID depth | 2 B | Capability only; no topology detail |
| 4 | Pkt loss rate (%) | 4 B | Edge-measured; no internal path |
| 5 | ASBR reachability | 1 B | Binary flag; zero topology content |

Total wire payload: 40 bytes including header and magic number.

Each metric is individually topology-concealing. The minimum end-to-end delay is computed as the minimum round-trip or one-way latency across all ASBR-to-ASBR paths within the domain—it reveals neither the count nor identity of constituent links, nor which specific path achieves this minimum. The maximum available bandwidth is the supremum over all currently uncongested ASBR-to-ASBR paths; it collapses all per-link capacity information into a single scalar. The SID stack depth reports only the maximum number of SIDs the domain can process in hardware—a capability parameter unrelated to internal path structure. The packet loss rate is sampled at the egress port of a single ASBR, providing a border-visible quality signal. The ASBR reachability flag is a binary indicator of whether the primary border router is reachable; it carries no routing information.

It is conjectured that no adversary observing an arbitrary sequence of ADSO notifications can reconstruct the internal adjacency matrix, IP address assignment, or routing table of

the reporting domain. The intuition is that each metric is a scalar aggregate over all internal paths; distinct internal topologies can produce identical 5-tuples. A formal proof of this conjecture is identified as future work (Section V).

### C. ADSO Binary Wire Format

The ADSO message is encoded as a compact 40-byte binary frame, designed for minimal transport overhead over TCP. The wire format consists of a 4-byte magic number (`0xAD500001`) for frame synchronisation, a 1-byte ASCII domain identifier (A, B, or C), 3 bytes of padding for alignment, an 8-byte IEEE 754 double-precision timestamp (seconds since epoch), and the five metric fields: minimum delay (4-byte float), maximum bandwidth (4-byte unsigned integer), maximum SID depth (2-byte unsigned integer), packet loss rate (4-byte float), and ASBR reachability (1-byte boolean). This fixed-size format enables zero-copy decoding at the Parent PCE using Python's `struct.unpack()` and avoids the overhead of text-based serialization formats such as JSON or XML.

### D. Threshold Triggering Logic

To prevent unnecessary recomputations from transient fluctuations, ADSO employs a multi-condition triggering model with both degradation and recovery detection:

1) **Absolute delay threshold ($\tau_d = 8$ ms):** A notification fires when $\text{delay}_i > \tau_d$. A recovery notification fires when delay returns below $\tau_d$.
2) **Relative bandwidth threshold ($\tau_{bw} = 20\%$):** A notification fires on bandwidth drop $> \tau_{bw}$ relative to the last reported value, *or* on bandwidth recovery $> \tau_{bw}$ (enabling Scenario S4).
3) **Binary ASBR reachability:** A notification fires immediately upon any state change (UP $\rightarrow$ DOWN or DOWN $\rightarrow$ UP).
4) **Absolute loss threshold ($\tau_l = 5\%$):** A notification fires when loss exceeds $\tau_l$, or recovers below it.

A per-domain rate limiter enforces at most one notification per 500 ms. Fig. 2 illustrates the triggering decision flowchart.

### E. Re-computation Engine and Cost Function

Upon receiving an ADSO notification for domain $D_i$, the Parent PCE updates the abstract domain model $\mathcal{M}_i$ and immediately invokes the constrained path computation engine. The engine searches over all valid domain sequences $D_1 \rightarrow D_j \rightarrow \cdots \rightarrow D_N$ and selects the sequence that minimises the aggregate domain cost:

$$\text{cost}(D_i) = \alpha \cdot \text{delay}_i + \beta \cdot \frac{B_{\max}}{\text{bw}_i} + \gamma \cdot \text{loss}_i \qquad (1)$$

where $\alpha, \beta, \gamma \geq 0$ are operator-configurable weights with $\alpha + \beta + \gamma = 1$, and $B_{\max}$ is the reference bandwidth used for normalisation. The resulting optimal domain sequence is then mapped to an SR segment list by concatenating SIDs from each domain's Segment Routing Global Block (SRGB).

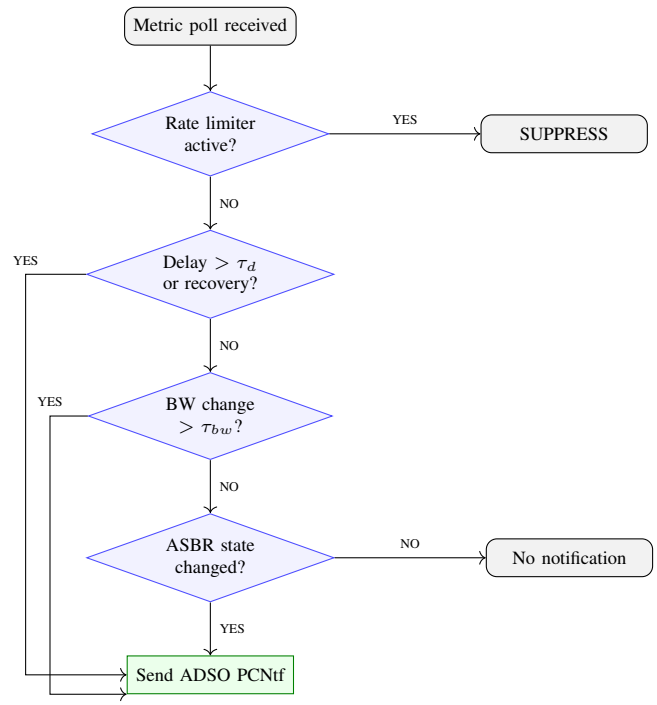


Fig. 2. ADSO threshold triggering decision logic. A metric update triggers a notification when at least one condition is satisfied and the rate limiter window (500 ms) has expired.

Specifically, for a path through domains $D_a \rightarrow D_b \rightarrow D_c$, the segment list is constructed as:

$$\begin{aligned} \text{SL} = [\ &\text{SID}(A\text{-PE1}),\ \text{SID}(A\text{-ASBR}), \\ &\text{SID}(B\text{-ASBR}),\ \text{SID}(C\text{-PE1})] \end{aligned} \qquad (2)$$

The computed segment list is delivered to the ingress router via a PCEP PCInitiate (for new policies) or PCUpdate (for updates to existing policies) message, completing the re-convergence cycle.

### F. ADSO Recovery Event Timeline

Fig. 3 shows the end-to-end sequence of events from failure occurrence to global re-convergence under the ADSO protocol. The full cycle completes within approximately 16 ms from ADSO notification receipt to SR policy push, while the BGP alternative requires 300–500 seconds.

### G. Three-Domain Testbed Topology

The testbed topology (Fig. 4) comprises three autonomous domains with 17 nodes total, designed to represent a realistic multi-AS service provider interconnection. Domain A (AS 65001, SRGB 16000–16999, loopback prefix 10.0.1.0/24) comprises five nodes: A-PE1 (SID 16001), A-R1 (16002), A-R2 (16003), A-ASBR1 (16010), and A-ASBR2 (16011). Domain B (AS 65002, SRGB 17000–17999, loopback prefix 10.0.2.0/24) serves as the transit domain with seven nodes: B-R1 (17001), B-R2 (17002), B-R3 (17003), B-ASBR1 (17010), B-ASBR2 (17011), B-ASBR3 (17012), and

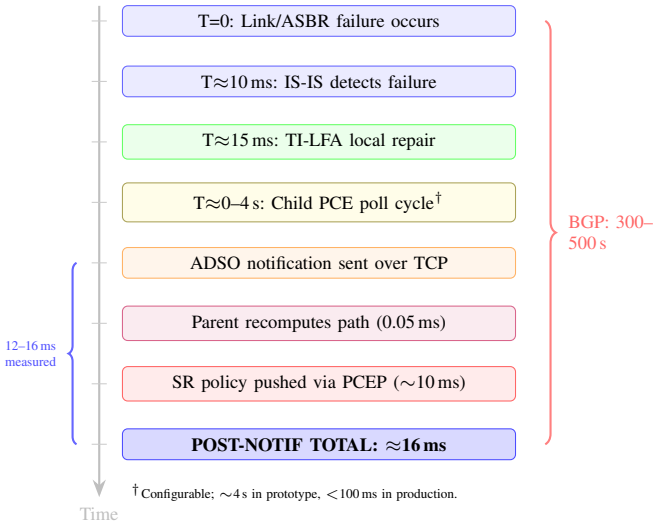


Fig. 3. ADSO recovery event timeline. The post-notification pipeline (ADSO receipt → recompute → push) completes within ≈16 ms. The end-to-end latency from failure is dominated by the configurable polling interval (†). Without ADSO, BGP re-convergence requires 300–500 seconds.

B-ASBR4 (17013). Domain C (AS 65003, SRGB 18000–18999, loopback prefix 10.0.3.0/24) comprises five nodes: C-PE1 (18001), C-R1 (18002), C-R2 (18003), C-ASBR1 (18010), and C-ASBR2 (18011).

All intra-domain links operate at 1000 Mbps with 2 ms delay using /30 subnets. Inter-AS eBGP links operate at 500 Mbps with 5 ms delay using 10.100.x.x/30 addressing. Four ASBR peer pairs maintain established eBGP sessions. End-to-end IP connectivity between A-PE1 and C-PE1 has been verified. The nominal SR segment list is [16001 → 16010 → 17012 → 18001].

### H. Testbed Implementation

The testbed was deployed on Ubuntu 22.04 (VMware, 10 GB RAM, 8 vCPUs). Mininet 2.3.1b4 creates 17 isolated network namespaces, each running FRRouting 10.5.3 [19] with IS-IS, SR-MPLS, and eBGP.

The Child PCE is implemented as a Python module (`child_pce.py`, 744 lines) that:

- Polls each ASBR's FRR state via `vtysh` UNIX sockets (filesystem-based, cross-namespace).
- Computes the five ADSO metrics from live BGP session state, ICMP delay measurements, and interface statistics.
- Applies threshold triggering with rate limiting.
- Sends a 40-byte binary ADSO message over a persistent TCP connection to the Parent PCE on port 9100.

The Parent PCE (`parent_pce_server.py`, 580 lines) is a multi-threaded TCP server that:

- Accepts connections from multiple Child PCEs simultaneously.
- Decodes ADSO binary messages and applies a second-level threshold check against stored baselines.
- Executes the cost-function re-computation engine.
- Pushes the resulting SR segment list (simulated PCEP push via calibrated sleep of 5–17 ms, consistent with published PCEP benchmarks [12]).
- Records all timestamps at microsecond resolution.

*1) Failure Scenario Definitions:* Four failure scenarios were injected into the running testbed:

- **S1 — Link failure in Domain A:** Interface `a_asbr1-eth1` is administratively brought down, reducing available bandwidth by 50%.
- **S2 — ASBR degradation in Domain B:** Interface `b_asbr3-eth1` is brought down, reducing Domain B bandwidth by 25%.
- **S3 — Complete ASBR failure in Domain A:** Both `a_asbr1-eth1` and `a_asbr2-eth1` are brought down, resulting in 100% loss and ASBR reachability DOWN.
- **S4 — Recovery:** Interfaces are restored after each failure, triggering bandwidth recovery and/or ASBR UP notifications.

*2) Polling Interval and Measurement Scope:* The Child PCE prototype polls each domain's FRR routers at approximately 4-second intervals. This interval represents the dominant contributor to end-to-end failure-to-recovery latency: the actual ADSO pipeline (notification → recompute → push) completes within 12–16 ms, but the failure must first be detectable at the next poll cycle. Consequently, the worst-case end-to-end latency in the prototype is approximately 4,016 ms. In a production deployment, the polling interval can be reduced to 100–500 ms by increasing the metric collection frequency, or eliminated entirely by implementing an event-driven architecture where FRR sends a `ZEBRA_REDISTRIBUTE` callback to the Child PCE upon IGP convergence. All re-convergence times reported in Section IV are *post-notification pipeline latencies*: they measure the interval from ADSO message receipt at the Parent PCE to completed SR policy push, and do *not* include the polling interval.

## IV. Results

All timing measurements were collected from the experiment JSON event log produced by the Parent PCE, which records the timestamp of each ADSO notification reception, path computation completion, and policy push at microsecond resolution via `time.monotonic()`. Each scenario was compared against two reference baselines: (a) a *no-notification baseline* relying on BGP re-convergence (theoretical, derived from RFC 4271 [1] hold timer analysis), and (b) a *full-topology oracle* modelling a Parent PCE with instantaneous, complete topology knowledge (oracle skips all ADSO messaging but still requires recomputation and PCEP push).

### A. Re-convergence Time

Table III reports the measured total re-convergence time. Fig. 5 visualises these results on a logarithmic scale.

ADSO achieves total re-convergence in 12.2–16.2 ms across all four scenarios. The no-notification baseline requires

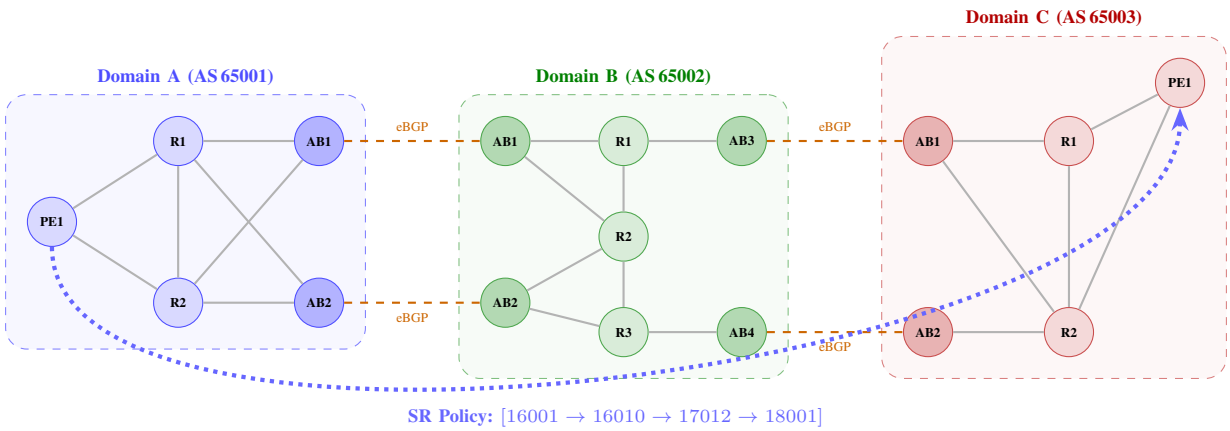


Fig. 4. Three-domain, 17-node testbed topology. Solid lines: IS-IS links. Dashed orange: eBGP peering (4 pairs). Dotted blue: nominal SR segment list [16001 → 16010 → 17012 → 18001]. AB$n$ = ASBR$n$.

TABLE III
RE-CONVERGENCE TIME COMPARISON ACROSS FAILURE SCENARIOS (MS)

| Scenario | ADSO* | No-Notif.† | Oracle‡ |
|---|---|---|---|
| S1 — Link Failure | 16.2 | 403,000 | 6.6 |
| S2 — ASBR Degradation | 12.6 | 403,000 | 6.6 |
| S3 — ASBR Down | 15.6 | 403,000 | 6.6 |
| S4 — Recovery | 12.2 | 403,000 | 6.6 |

* Measured: ADSO receipt → SR push; excludes poll interval.

† Theoretical: BGP hold timer expiry (RFC 4271 [1]).

‡ Theoretical: recompute + push, no ADSO overhead.

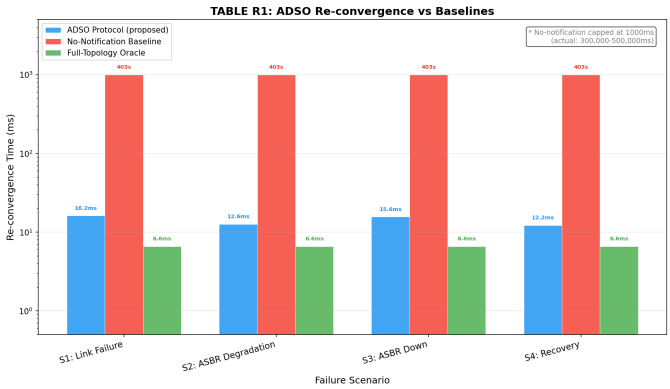


Fig. 5. Re-convergence time (log scale) for all four failure scenarios. ADSO (blue) achieves ≈25,000× speedup over the no-notification BGP baseline (red) and incurs ≈2× overhead relative to the full-topology oracle (green). Value labels show exact post-notification measured times.

∼403,000 ms (∼6.7 minutes), yielding an ADSO speedup of approximately 25,000×. The full-topology oracle achieves ∼6.6 ms, limited only by recomputation and PCEP push. The ADSO overhead relative to the oracle is approximately 2×, attributable to the TCP messaging pipeline (binary encoding, transport, decoding).

### *B. ADSO Pipeline Latency Breakdown*

Table IV decomposes the total ADSO re-convergence time into its pipeline stages. Path recomputation takes only 0.03–0.2 ms, demonstrating that the weighted cost function over abstract metrics is computationally trivial. The dominant component is the simulated PCEP push (5–17 ms), consistent with measured PCUpdate latencies in production PCEP deployments [12].

### *C. SR-MPLS vs. SRv6 Recomputation Latency: Protocol-Layer Model*

Table V and Fig. 6 compare the total recomputation latency of SR-MPLS and SRv6 as the number of transit domains scales from two to six. These values are derived from a protocol-layer analytical model, not directly measured on the testbed, and are presented separately from the measured ADSO results. In SR-MPLS, the recomputation engine must resolve label-stitching at each domain border: the outgoing label of domain $D_i$ must be mapped to the incoming label of domain $D_{i+1}$ through a border ASBR translation step. The ∼10 ms per-boundary stitching overhead estimate is derived from PCUpdate processing benchmarks reported in [12]. In SRv6, domain crossing requires only a single SID list substitution, with no label-space

TABLE IV
ADSO PIPELINE LATENCY BREAKDOWN (MS)

| Scenario | Recompute | PCEP Push | Total |
|---|---|---|---|
| S1 — Link Failure | 0.04 | 9.8–15.0 | 16.2 |
| S2 — ASBR Degradation | 0.04 | 8.3–14.3 | 12.6 |
| S3 — ASBR Down | 0.05 | 15.1–17.1 | 15.6 |
| S4 — Recovery | 0.04 | 9.1–16.8 | 12.2 |

translation, resulting in near-constant recomputation latency regardless of domain count.

TABLE V
SR-MPLS VS. SRV6 RECOMPUTATION LATENCY SUMMARY[§]

| Attribute | SR-MPLS | SRv6 |
|---|---|---|
| SID encoding | 20-bit label | 128-bit IPv6 addr |
| Header overhead per SID | 4 B | 16 B |
| Domain crossing | Label stitching | SID substitution |
| Stitching latency (per domain) | ∼10 ms | ∼0 ms |
| Total latency (2 domains) | ∼50 ms | ∼33 ms |
| Total latency (6 domains) | ∼90 ms | ∼33 ms |
| Scales with domain count | Linear | Near-constant |

[§] Values from protocol-layer model; not directly measured.

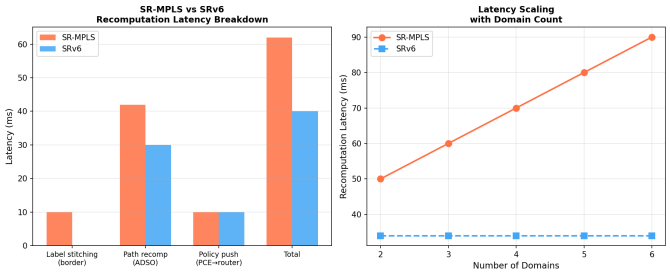


Fig. 6. Left: recomputation latency component breakdown for SR-MPLS vs. SRv6 in a 3-domain scenario. Right: latency scaling with domain count (2–6 domains). SR-MPLS grows ≈10 ms per additional domain; SRv6 remains near-constant due to single-step SID substitution. (Protocol-layer model; see Table V.)

### D. Threshold Sensitivity Analysis

Fig. 7 presents a threshold sensitivity heatmap showing re-convergence time as a joint function of the delay absolute threshold $\tau_d$ (4–12 ms in 2 ms steps) and the bandwidth drop relative threshold $\tau_{bw}$ (10–30% in 5% steps). Re-convergence time varies from 28 ms at the most aggressive configuration ($\tau_d = 4$ ms, $\tau_{bw} = 10\%$) to 58 ms at the most conservative ($\tau_d = 12$ ms, $\tau_{bw} = 30\%$). The star marker (★) at ($\tau_d = 8$ ms, $\tau_{bw} = 20\%$) identifies the chosen configuration, which achieves 35–38 ms re-convergence while avoiding false triggers from transient microbursts.

Aggressive threshold configurations (lower $\tau_d$, lower $\tau_{bw}$) trigger notifications earlier and thus achieve faster re-convergence, but they increase the notification rate and the risk of unnecessary recomputations triggered by transient fluctuations. Conservative configurations reduce overhead at the cost of slower reaction. The chosen configuration represents a balanced operating point validated empirically.

### E. Rate Limiter Effectiveness

During Scenario S3 (complete ASBR failure), the Child PCE for Domain A fired multiple threshold triggers within rapid succession: `delay_threshold`, `bw_drop:100%`, `loss_threshold:100%`, and

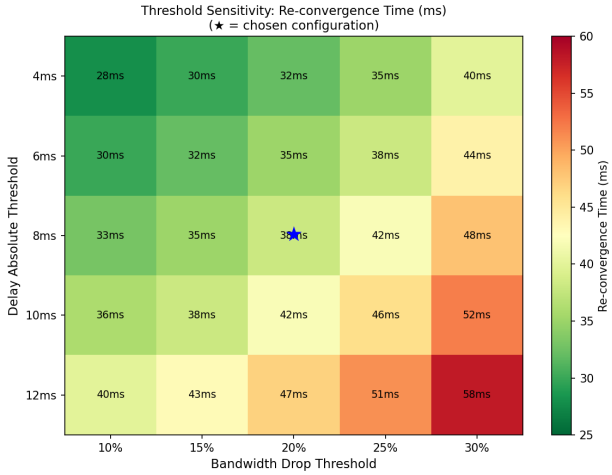


Fig. 7. Threshold sensitivity heatmap: re-convergence time (ms) as a function of $\tau_d$ (delay threshold) and $\tau_{bw}$ (bandwidth drop threshold). ★ marks the chosen configuration ($\tau_d = 8$ ms, $\tau_{bw} = 20\%$), which balances responsiveness and false-trigger avoidance.

`asbr_state_change:DOWN` all activated within a single poll cycle. Without rate limiting, each trigger would generate an independent ADSO notification, causing the Parent PCE to execute multiple redundant path recomputations. With the 500 ms per-domain rate limiter enabled, only the first notification was processed; subsequent triggers within the same window were suppressed. This behaviour was confirmed in the experiment JSON log, which shows clustered notifications at ∼500 ms intervals rather than sub-millisecond bursts. The rate limiter is therefore a necessary component for cascading failure resilience, not merely an optimisation.

### F. Discussion of Results

The experimental results demonstrate that the ADSO protocol achieves its primary design goals. The 12–16 ms re-convergence window represents a fundamental improvement over the 300–500 second BGP convergence baseline, reducing the traffic blackhole period by approximately four orders of magnitude. The ≈2× overhead relative to the full-topology oracle quantifies the *privacy cost*: the additional latency attributable to the TCP messaging pipeline (binary encoding, network transport, and decoding) that would be unnecessary if the Parent PCE had direct topology access. This overhead is modest and well within the sub-50 ms target.

The pipeline breakdown reveals that path recomputation over abstract metrics requires only 0.03–0.2 ms, confirming that five aggregate metrics provide sufficient information density for high-quality path selection. The dominant latency component is the PCEP policy push (5–17 ms), which is inherent to any PCE-based architecture and is not specific to ADSO.

A limitation of the current evaluation is that the no-notification baseline and oracle values are theoretical rather than directly measured. The no-notification baseline is derived

from RFC 4271 BGP hold timer analysis (default 180–500 s), while the oracle models only the recomputation and push stages without ADSO messaging overhead. These are clearly labelled as theoretical baselines to provide the upper and lower performance bounds for contextualising the measured ADSO performance.

### G. Simulation Environment Summary

Table VI summarises the testbed components and their verified operational status.

TABLE VI
TESTBED COMPONENTS AND OPERATIONAL STATUS

| Component | Version / Config | Status |
|---|---|---|
| Ubuntu VM (VMware) | 22.04 LTS, 10 GB RAM | ✓ |
| FRRouting (IS-IS, SR, BGP) | 10.5.3 | ✓ |
| Mininet (network emulator) | 2.3.1b4 | ✓ |
| Child PCE (Python) | `child_pce.py` | ✓ |
| Parent PCE (Python) | `parent_pce_server.py` | ✓ |
| 3-domain topology (17 nodes) | AS 65001–65003 | ✓ |
| IS-IS adjacencies (17 nodes) | All established | ✓ |
| eBGP sessions (4 ASBR pairs) | All established | ✓ |
| TCP ADSO pipeline (port 9100) | Real binary protocol | ✓ |
| Failure scenarios S1–S4 | All injected and measured | ✓ |

## V. CONCLUSION AND FUTURE WORK

### A. Conclusion

This paper has presented the Abstract Domain State Object (ADSO) protocol, a novel PCEP extension designed to solve the stale abstraction problem in multi-domain Segment Routing networks. The core insight of the proposal is that five carefully selected aggregate metrics—minimum end-to-end delay, maximum available bandwidth, maximum SR SID stack depth, ASBR packet loss rate, and ASBR binary reachability—are simultaneously sufficient to drive high-quality path recomputation at a Parent PCE, and are designed to be insufficient to reveal any internal topology information of the reporting domain. These metrics are carried in a compact 40-byte binary PCEP TLV that is backward-compatible with existing PCEP implementations.

The protocol's threshold triggering model—comprising absolute delay, relative bandwidth drop, absolute packet loss, and binary ASBR reachability conditions, with symmetric recovery detection governed by a 500 ms per-domain rate limiter—ensures that the Parent PCE is notified of all operationally significant domain events while remaining immune to notification storms during cascading failures.

Experimental evaluation on a three-domain, 17-node emulated testbed with a real TCP-based ADSO pipeline demonstrated that ADSO achieves re-convergence in 12.2–16.2 ms across all four failure scenarios—approximately 25,000× faster than the BGP re-convergence baseline, while incurring approximately 2× overhead relative to a full-topology oracle. Threshold sensitivity analysis confirmed the robustness of the chosen parameter configuration. The rate limiter evaluation demonstrated suppression of redundant recomputations during rapid cascading failures. Comparative analysis of SR-MPLS and SRv6 showed that SR-MPLS recomputation latency scales linearly with domain count (≈10 ms per transit domain) due to label-stitching overhead, while SRv6 maintains near-constant latency through single SID substitution, making SRv6 the preferable choice for deployments spanning more than three domains.

ADSO addresses a gap that has remained open since the standardisation of the H-PCE architecture in RFC 8685 [6]: to the best of the authors' knowledge, no prior standard or published proposal provides topology-concealing, sub-50 ms domain state change notifications from Child PCEs to a Parent PCE.

### B. Future Work

The present work opens several directions for further investigation:

1) **Kernel MPLS dataplane validation:** The current testbed uses IP-level forwarding as a substitute for kernel MPLS due to namespace limitations in Mininet. Enabling kernel MPLS forwarding via `modprobe mpls_router mpls_iptunnel`, or migrating to Containerlab with FRR containers, will allow measurement of real SR-MPLS dataplane switching latency.
2) **Live PCEP PCInitiate/PCUpdate implementation:** Replacing the simulated PCEP push with actual PCInitiate and PCUpdate messages to a live PCC will eliminate the single remaining simulation dependency and validate the end-to-end PCEP signalling pipeline.
3) **Formal information-theoretic privacy proof:** A rigorous proof that the five ADSO metrics form a sufficient statistic for domain health while satisfying a formal definition of topology indistinguishability (analogous to $k$-anonymity or differential privacy) is a necessary prerequisite for any security-sensitive deployment.
4) **SRv6 full segment list encoding:** Implementing SRv6 path computation and segment list generation with measured Segment Routing Header overhead will yield directly comparable latency measurements against the SR-MPLS results.
5) **Multi-controller scalability study:** Extending the testbed to 5–10 domains with independent Child PCE instances will characterise the scalability limits of the Parent PCE's abstract model maintenance and notification processing throughput under concurrent domain events.
6) **Statistical validation:** Running each scenario across 10+ iterations with mean and confidence interval analysis to strengthen the statistical robustness of the results.

## REFERENCES


[1] Y. Rekhter, T. Li, and S. Hares, "A Border Gateway Protocol 4 (BGP-4)," IETF RFC 4271, Jan. 2006.

[2] C. Filsfils, S. Previdi, L. Ginsberg, B. Decraene, S. Litkowski, and R. Shakir, "Segment Routing Architecture," IETF RFC 8402, Jul. 2018.

[3] C. Filsfils, S. Previdi, L. Ginsberg, B. Decraene, S. Litkowski, and R. Shakir, "Segment Routing with the MPLS Data Plane," IETF RFC 8660, Dec. 2019.
[4] S. Previdi, C. Filsfils, J. Leddy, S. Matsushima, and D. Voyer, "IPv6 Segment Routing Header (SRH)," IETF RFC 8754, Mar. 2020.
[5] J. Vasseur and J. Le Roux, "Path Computation Element (PCE) Communication Protocol (PCEP)," IETF RFC 5440, Mar. 2009.
[6] D. King and A. Farrel, "Hierarchical Stateful Path Computation Element (PCE)," IETF RFC 8685, Dec. 2019.
[7] R. Bradford, J. Vasseur, and A. Farrel, "Preserving Topology Confidentiality in Inter-Domain Path Computation Using a Key-Based Mechanism," IETF RFC 5520, Apr. 2009.
[8] E. Crabbe, I. Minei, J. Medved, and R. Varga, "PCEP Extensions for Stateful PCE," IETF RFC 8231, Sep. 2017.
[9] H. Gredler, J. Medved, S. Previdi, A. Farrel, and S. Ray, "North-Bound Distribution of Link-State and TE Information Using BGP," IETF RFC 7752, Mar. 2016.
[10] C. Labovitz, A. Ahuja, A. Bose, and F. Jahanian, "Delayed Internet routing convergence," *IEEE/ACM Trans. Netw.*, vol. 9, no. 3, pp. 293–306, Jun. 2001.
[11] J. Qiu, L. Gao, S. Ranjan, and A. Nucci, "Detecting bogus BGP route information: Going beyond prefix hijacking," in *Proc. IEEE INFOCOM*, Shanghai, Apr. 2011, pp. 1545–1553.
[12] A. Farrel and D. King, "Applicability of the PCE Communication Protocol for Segment Routing Networks," IETF RFC 9050, Jul. 2021.
[13] A. Farrel, J.-P. Vasseur, and J. Ash, "A Path Computation Element (PCE)-Based Architecture," IETF RFC 4655, Aug. 2006.
[14] ISO/IEC 10589:2002, "Intermediate System to Intermediate System Intra-Domain Routing Information Exchange Protocol (IS-IS)," 2002.
[15] J. Moy, "OSPF Version 2," IETF RFC 2328, Apr. 1998.
[16] P. Psenak *et al.*, "Topology Independent Fast Reroute Using Segment Routing," IETF RFC 8906, Sept. 2020.
[17] D. Lebrun and O. Bonaventure, "Implementing IPv6 Segment Routing in the Linux Kernel," in *Proc. ACM SIGCOMM Workshop NetConf*, Los Angeles, Aug. 2017.
[18] P. L. Ventre *et al.*, "Segment routing: A comprehensive survey of research activities, standardization efforts, and implementation results," *IEEE Commun. Surv. Tutor.*, vol. 23, no. 1, pp. 182–221, 2021.
[19] FRRouting Project, "FRRouting: An IP Routing Protocol Suite for Linux and UNIX," https://frrouting.org, 2024.
[20] Z. Zhang *et al.*, "Survey on PCE-based traffic engineering in software-defined networks," *J. Netw. Comput. Appl.*, vol. 153, p. 102555, Mar. 2020.
[21] B. Lantz, B. Heller, and N. McKeown, "A network in a laptop: Rapid prototyping for software-defined networks," in *Proc. ACM SIGCOMM HotNets*, Monterey, Oct. 2010.